Tensor Omics (TOX)

Analyze gene expression data and evolutionary relationships between genes

User's Guide

Vivian Bass[1], Anna Beketova[1], Luka Faensen[1], Asis Hallab[1], Aaron Schröder[1], Franz-Eric Sill[1]

[1]University of Applied Sciences Bingen, Germany

Last revised August 12, 2025

Contents

1 Tensor Omics Method and Theory

The Tensor Omics (TOX) tool implements a streamlined pipeline for analyzing gene expression data and evolutionary relationships between genes. The pipeline integrates tools for data normalization, and geometric analysis of gene expression vectors. This document outlines the current implementation details and provides the mathematical formulation of the algorithms used in the prototype.

1.1 From Raw Data to Expression Vectors

The starting point for all downstream analyses is a table of gene-wise expression measurements across a set of biological samples, such as tissues, developmental stages, or experimental conditions. These measurements may originate from transcriptomic, proteomic, or metabolomic experiments, typically normalized to account for sequencing depth and technical variation.

What is Gene Expression Data? For each gene, a real-valued quantity is recorded that reflects its biological activity in a given sample. In the case of transcriptomics, this is often the abundance of messenger RNA (mRNA) molecules transcribed from that gene, estimated via sequencing and mapped back to the gene's known sequence. The result is a non-negative real number proportional to the gene's transcriptional activity in that sample. After normalization and transformation, these values can be treated as coordinates in a real vector space.

Thus, each gene is represented as a vector $\vec{v} \in \mathbb{R}^n$, where n is the number of biological samples (e.g., tissues or time points), and each component $v_i \geq 0$ corresponds to the expression level in sample i. These vectors encode the distribution of gene activity across conditions and form the geometric basis for all downstream analyses.

Note on Stress Responses. In certain studies, a subset of the samples may correspond to stress conditions. Here, "stress" refers to any external or internal perturbation—such as mechanical damage, heat shock, pathogen infection, or nutrient deprivation—that triggers measurable changes in gene expression. These are often analyzed relative to baseline conditions using log fold-change transformations, such that the resulting vectors reflect differential expression patterns, including up- and downregulation across dimensions.

1.2 Normalization of Expression Values

Raw gene expression data is normalized in the following steps:

1. Division by gene-wise scale: this normalizes each gene's expression vector to unit root mean square (RMS) length across samples by computing the root mean square:

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n} x_i^2}$$

2. Quantile normalization across samples to reduce global expression biases.

3. Log-transformation: $x \mapsto \log(1+x)$ to stabilize variance and suppress outliers.

4. For stress response studies, log fold-changes are computed:

$$\Delta_{\text{stress}} = \log(1 + x_{\text{stress}}) - \log(1 + x_{\text{control}})$$

This effectively makes each stress related axis show the fold change in gene expression under stress, e.g. "drought in root divided by control root".

1.3 Calculation of family Centroids

To summarize the expression profile of each family, Tensor Omics computes a centroid vector for every group. The centroid represents the average expression pattern of all genes assigned to the family across all conditions or tissues.

There are two modes for selecting the set S_g of genes to include in the centroid:

- **Full mode:** All genes assigned to the family are used to compute the centroid.
- **Orthologs-only mode:** Only genes explicitly marked as orthologs are included in the centroid calculation. This mode is useful for focusing on conserved gene relationships across species.

1.4 Euclidean Distance Between Expression Vectors

The Euclidean distance is used throughout Tensor Omics to quantify the geometric difference between two gene expression vectors, or between a gene and its family centroid. Given two vectors $\vec{v}_1, \vec{v}_2 \in \mathbb{R}^n$, the Euclidean distance is defined as:

$$d(\vec{v}_1, \vec{v}_2) = \sqrt{\sum_{i=1}^{n} (v_{1,i} - v_{2,i})^2}$$

This metric measures the straight-line distance between two points in n-dimensional space. In the context of gene expression, it reflects the overall magnitude of difference in expression profiles across all samples or conditions.
For example, the distance between a paralog's expression vector and the centroid of its gene family is used to detect outlier genes and to quantify the degree of divergence.

1.5 Outlier Detection Based on Family-Scaled Distances

To robustly identify genes with divergent expression profiles within their families, Tensor Omics implements an outlier detection procedure based on family-scaled Euclidean distances. The process consists of the following steps:

1. **Compute Euclidean Distances:** For each gene, calculate the Euclidean distance d_i between its expression vector and the centroid of its assigned family (see Section 1.4).
2. **Estimate Family-Specific Scaling Factors:** For each family f, collect the set of distances $\{d_i\}$ for all genes assigned to f. Compute the median and standard deviation of these distances. To account for family size and heterogeneity, a LOESS (locally weighted regression) smoothing is applied to the relationship between family median and standard deviation, yielding a robust scaling factor s_f for each family.
3. **Relative Distance Index (RDI):** For each gene i in family f, compute the Relative Distance Index (RDI):
$$\mathrm{RDI}_i = \frac{|d_i|}{s_f}$$

This normalizes the gene's distance by the expected intra-family variability, making the metric comparable across families of different sizes and variances.

4. **Outlier Detection:** Genes are flagged as outliers if their RDI exceeds a chosen percentile threshold (typically the 95th percentile) of the empirical RDI distribution. That is, gene i is an outlier if $\mathrm{RDI}_i \geq T$, where T is the threshold value corresponding to the desired percentile.

This approach ensures that outlier detection is adaptive to the structure and variability of each gene family, and is robust to differences in family size and dispersion. The LOESS-based scaling is particularly important for families with few members.

1.6 Tissue Versatility and Specificity

Tissue Versatility quantifies how uniformly a gene is expressed across all tissues (or, more generally, axes of the expression space). It is defined as the angle between a gene's expression vector $\vec{v}$ and the space diagonal $\vec{d} = (1, 1, \ldots, 1)$ in the positive orthant. The smaller this angle, the greater the tissue (axis) versatility of the gene's expression.

Interpretation:

- A lower angle means the gene is more evenly expressed across all tissues (high versatility).
- A larger angle means the gene is active in only a few tissues (high specificity).

Dimensionality Dependence

The maximum angle to the diagonal in the positive orthant depends on the number of tissues n. The cosine of this maximum angle (when a gene is expressed in only one tissue) is:

$$\cos(\varphi_{\min}) = \frac{1}{\sqrt{n}}$$

Normalized Tissue Versatility

To make versatility comparable across different numbers of tissues, we define a normalized version:

$$\text{Tissue Versatility} = \frac{1 - \cos(\varphi)}{1 - \frac{1}{\sqrt{n}}}$$

This scales values into the interval $[0, 1]$:

- 0: perfectly uniform expression (maximum versatility)
- 1: expression in only one tissue (maximum specificity)

Tissue Specificity

Tissue specificity is defined as the complement of tissue versatility:

$$\text{Tissue Specificity} = 1 - \text{Tissue Versatility}$$

Values close to 1 indicate high tissue-specificity; values close to 0 indicate broad expression.

2 Error Handling

In case you encounter errors while using TOX, the following error codes may help you identify the issue. The error codes are grouped into categories based on their nature, such as I/O errors, format validation errors, memory errors, and internal errors.

Table 1: TOX Error Codes Reference

`Error Code`	**Description**
`0`	Success
1xx: I/O / File Reading Errors	
`101`	Could not open file.
`102`	Could not read magic number.
`103`	Could not read array type code.
`104`	Could not read number of dimensions.
`105`	Could not read array dimensions.
`106`	Could not read character length.
`107`	Could not read array data.
2xx: Format / Input Validation Errors	
`200`	Invalid format detected (e.g., magic mismatch or corrupt format).
`201`	Invalid input provided.
`202`	Empty input arrays provided (e.g., zero-length arrays).
`203`	Dimension mismatch detected (shape inconsistencies).
`204`	NaN or Inf found in input data where not allowed.
`205`	Unsupported data type encountered (e.g., complex types).
3xx: Memory Errors	
`301`	Memory allocation failed.
`302`	Null pointer reference encountered.
5xxx: Fortran Runtime / Unit State Errors	
`5002`	Fortran runtime error: unit not open or not connected.
9xxx: Internal / Unknown Errors	
`9001`	Internal error: unexpected state or invariant violated.
`9999`	Unknown error (fallback for unexpected errors).

3 Implementation Notes